

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 7

8 Further Graphics (aco41)

This question is on rendering a scene via ray tracing and the rendering equation.

Ray tracing for rendering

- (a) You are tasked with rendering a video of a scene via ray tracing. Only the camera is moving, the rest of the scene is static, the light sources emit light of a single wavelength, and surfaces do not emit light. You are allowed to shoot one ray per pixel and do pre-computation for a rotation and position of the camera. For each of the following cases, explain if this is possible, and if so, describe what you would pre-compute and store in single-channel textures, and how you would finally render the scene.
- (i) Local illumination and a single BRDF (bidirectional reflectance distribution function) for all surfaces. [3 marks]
- (ii) Global illumination and diffuse reflection for all surfaces. [3 marks]

Answer:

- (i) This case requires computing the integral $L_o(\mathbf{x}, \vec{\omega}_o) = \int_{H^2} f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o) L_i(\mathbf{x}, \vec{\omega}_i) \cos \theta_i d\vec{\omega}_i$, where the incoming light is only from light sources. As the integral depends on the viewing direction, which is changing as we move the camera, it is not possible to pre-compute and store the result in a single-channel texture and thus not possible to compute the colour of a pixel via a single ray in general for a reasonably accurate rendering. If accuracy is not important, a single value $L_i(\mathbf{x}, \vec{\omega}_i) \cos \theta_i$ for one $\vec{\omega}_i$ can be computed and stored. At rendering time, this is used to approximate the integral with a single sample.
- (ii) For diffuse reflection, we can write $L_o(\mathbf{x}, \vec{\omega}_o) = f_r(\mathbf{x}) \int_{H^2} L_i(\mathbf{x}, \vec{\omega}_i) \cos \theta_i d\vec{\omega}_i$ for the outgoing light at each scene point. As the integral is independent of the viewing direction and as nothing except the camera is changing in the scene, we can pre-compute the integral and store it in object textures as a single-channel texture. At rendering time, we shoot a ray for a given pixel, see where it intersects the geometry, and compute $f_r(\mathbf{x}) I(\mathbf{x})$, where $I(\mathbf{x})$ is the pre-computed integral.
-

Importance-sampling for ray tracing

- (b) We are rendering a scene via path tracing and approximating integrals with importance-sampling.
- (i) Assume you can importance-sample with the exact incoming radiance. Write the resulting approximation of outgoing radiance at any point and direction with N samples. What is the length (number of segments) of the light path to compute this approximation? Why is this length finite? [5 marks]
- (ii) There is only a single point light source in a scene, only light paths of length three are allowed, and no shadow rays are allowed. What would be your importance-sampling strategy, and why at the first and second intersection points for a given path? [4 marks]
-

Answer:

- (i) Importance-sampling with the incoming radiance means the term L_i in the sum approximating the rendering equation integral cancels and we get $L_o(\mathbf{x}, \vec{\omega}_o) \approx L_e(\mathbf{x}, \vec{\omega}_o) +$

$\frac{1}{N} \sum_j f_r(\mathbf{x}, (\vec{\omega}_i)_j, \vec{\omega}_o) \cos(\theta_i)_j$, where $(\cdot)_j$ denotes the j -th sample. [2 marks] This sum can be computed without shooting further rays and hence a light path of length one, i.e. one segment from the sensor to the intersection point, is sufficient. [1 mark] Normally in global illumination, we need light paths of infinite length. In this case, we make the unreasonable assumption that we can know and sample from incoming radiance. This is in practice not possible as it requires knowing how much light is coming from other points, which needs in turn to be computed via solving the recursive rendering equation. [2 marks]

- (ii) At the first intersection point, we can importance-sample according to the BRDF (bidirectional reflectance distribution function) or the multiplication of it with the cosine term. [2 marks] At the second intersection point in the light path, we know that we need to terminate the path after shooting one final ray. In this case, we can shoot a ray towards the point light source and see if the ray will reach the point light source or will be occluded by any other surfaces. [2 marks]

Ray tracing with constraints

- (c) For a given scene, assume the rendering equation can be reduced to $L_o(\mathbf{x}, \vec{\omega}_o) = c \int_{H^2} V(\mathbf{x}, \vec{\omega}_i) \cos \theta_i d\vec{\omega}_i$ at surface points $\mathbf{x}$, where V is the (binary) visibility function.

- (i) What are the properties of the scene that make this reduction possible? [3 marks]
- (ii) Assuming a single object and ignoring inter-reflections for that object, what is the gradient of $L_o(\mathbf{x}, \vec{\omega}_o)$ with respect to $\mathbf{x}$? [2 marks]

Answer:

- (i) The surfaces do not emit light [1 mark], all surfaces have diffuse reflection [1 mark], incoming light in the scene is equal at all surface points and in all directions [1 mark].
- (ii) In this case, the visibility term will drop and we are left with a constant L_o and thus the gradient is zero. [2 marks]
-